

Padded mdspan layouts

Document #: P2642
Date: 2023-01-15
Project: Programming Language C++
LEWG
Reply-to: Mark Hoemmen (NVIDIA)
<mhoemmen@nvidia.com>
Christian Trott (Sandia National Laboratories)
<crtrott@sandia.gov>
Damien Lebrun-Grandie (Oak Ridge National Laboratory)
<lebrungrandt@ornl.gov>
Malte Förster (NVIDIA)
<mfoerster@nvidia.com>
Jiaming Yuan (NVIDIA)
<jiamingy@nvidia.com>

Contents

- 1 Authors
- 2 Revision history
 - 2.1 Revision 0
 - 2.2 Revision 1
 - 2.3 Revision 2
- 3 Proposed changes and justification
 - 3.1 Summary of proposed changes
 - 3.2 Two new mdspan layouts
 - 3.2.1 Optimizations over `layout_stride`
 - 3.2.2 New layouts unify two use cases
 - 3.2.3 Design change from R0 to R1
 - 3.2.4 Padding stride equality for layout mapping conversions
 - 3.2.5 New layout mapping constructors in R2
 - 3.2.6 Conversion from `layout_left` to `layout_left_padded`
 - 3.2.7 Conversion from `layout_stride` to `layout_left_padded`
 - 3.3 Integration with `submdspan`
 - 3.3.1 `layout_left_padded` and `layout_left` cases

- 3.3.2 `layout_right_padded` and `layout_right` cases
- 3.4 Examples
 - 3.4.1 Directly call C BLAS without checks
 - 3.4.2 Overaligned access
- 3.5 Alternatives
- 3.6 Implementation experience
- 3.7 Desired ship vehicle
- 4 Wording
 - 4.1 Class template `layout_left_padded::mapping` [`mdspan.layout.left_padded`]
 - 4.2 Class template `layout_right_padded::mapping` [`mdspan.layout.right_padded`]
 - 4.3 Layout specializations of `submdspan_mapping` [`mdspan.submdspan.mapping`]
 - 4.4 Layout specializations of `submdspan_offset` [`mdspan.submdspan.offset`]

§ 1 Authors

- Mark Hoemmen (mhoemmen@nvidia.com) (NVIDIA)
- Christian Trott (crtrott@sandia.gov) (Sandia National Laboratories)
- Damien Lebrun-Grandie (lebrungrandt@ornl.gov) (Oak Ridge National Laboratory)
- Malte Förster (mfoerster@nvidia.com) (NVIDIA)
- Jiaming Yuan (jiamingy@nvidia.com) (NVIDIA)

§ 2 Revision history

§ 2.1 Revision 0

Revision 0 submitted 2022-09-14.

§ 2.2 Revision 1

Revision 1 submitted 2022-10-15.

- Change padding stride to function as an overalignment factor if less than the extent to pad. Remove mapping constructor that takes `extents_type` and `extents<index_type, padding_stride>`, because the latter may not be the actual padding stride.
- Make converting constructors from `layout_{left,right}_padded::mapping` to `layout_{left,right}::mapping` use Mandates rather than Constraints to check compile-time stride compatibility.

- Mandate that `layout_{left,right}_padded::mapping`'s actual padding stride, if known at compile time, be representable as a value of type `index_type` (as well as of type `size_t`, the previous requirement).
- Add section explaining why we don't permit conversion from more aligned to less aligned.
- Fixed typos in Wording
- Fix formatting in non-Wording, and add links for BLAS and LAPACK

§ 2.3 Revision 2

Revision 2 to be submitted 2023-01-15.

- Rebase atop P2630R2 (not R0).
- Fix synopsis declaration of `layout_{left,right}_padded` to declare the mapping as well
- Add that each specialization of `layout_{left,right}_padded` meets the layout mapping policy requirements and is a trivial type
- Simplify `layout_{left,right}_padded::mapping` constructor conditions
- Add default constructors for `layout_{left,right}_padded::mapping; =default` for *actual-padding-stride* != `dynamic_extent`, not `=default` otherwise
- Add converting constructors from `layout_(left,right)::mapping` to `layout_(left,right)_padded::mapping`
- Add converting constructors from `layout_stride::mapping` to `layout_(left,right)_padded::mapping`
- Add `operator==` to `layout_{left,right}_padded::mapping`
- In Remarks for the existing constructor `layout_stride::mapping(const StridedLayoutMapping&)`, add `layout_left_padded` and `layout_right_padded` to the list of the layouts in the expression inside `explicit`
- Reformat from Bikeshed to Pandoc

§ 3 Proposed changes and justification

§ 3.1 Summary of proposed changes

We propose two new `mdspan` layouts, `layout_left_padded` and `layout_right_padded`. These layouts support two use cases:

1. array layouts that are contiguous in one dimension, as supported by commonly used libraries like the [BLAS](#) (Basic Linear Algebra Subroutines; see P1417 and P1674 for historical overview and references) and [LAPACK](#) (Linear Algebra PACKage); and
2. “padded” storage for overaligned access of the start of every contiguous segment of the array.

We also propose changing `submdspan` of a `layout_left` resp. `layout_right` `mdspan` to return `layout_left_padded` resp. `layout_right_padded` instead of `layout_stride`, when the slice arguments permit it.

§ 3.2 Two new `mdspan` layouts

The two new `mdspan` layouts `layout_left_padded` and `layout_right_padded` are strided, unique layouts. If the rank is zero or one, then the layouts behave exactly like `layout_left` resp. `layout_right`. If the rank is two or more, then the layouts implement a special case of `layout_stride` where only one stride may differ from the extent that in `layout_left` resp. `layout_right` would completely define the stride. We call that stride the *padding stride*, and the extent that in `layout_left` resp. `layout_right` would define it the *extent to pad*. The padding stride of `layout_left_padded` is `stride(1)`, and the extent to pad is `extent(0)`. The padding stride of `layout_right_padded` is `stride(rank() - 2)`, and the extent to pad is `extent(rank() - 1)`. All other strides of `layout_left_padded` are the same as in `layout_left`, and all other strides of `layout_right_padded` are the same as in `layout_right`.

§ 3.2.1 Optimizations over `layout_stride`

The two new layouts offer the following optimizations over `layout_stride`.

1. They guarantee at compile time that one extent always has stride-1 access. While `layout_stride`’s member functions are all `constexpr`, its mapping constructor takes the strides as a `std::array` with `rank()` size.
2. They do not need to store any strides if the padding stride is known at compile time. Even if the padding stride is a run-time value, these layouts only need to store the one stride value (as `index_type`). The `layout_stride::mapping` class must store all `rank()` stride values.

§ 3.2.2 New layouts unify two use cases

The proposed layouts unify two different use cases:

1. overaligned access to the beginning of each contiguous segment of elements, and
2. representing exactly the data layout assumed by the General (GE) matrix type in the BLAS’ C binding.

Regarding (1), an appropriate choice of padding can ensure any desired overalignment of the beginning of each contiguous segment of elements in an `mdspan`, as long as the entire

memory allocation has the same overalignment. This is useful for hardware features that require or perform better with overaligned access, such as SIMD (Single Instruction Multiple Data) instructions.

Regarding (2), the padding stride is the same as BLAS’ “leading dimension” of the matrix (LDA) argument. Unlike `layout_left` and `layout_right`, any subview of a contiguous subset of rows and columns of a rank-2 `layout_left_padded` or `layout_right_padded` `mdspan` preserves the layout. For example, if `A` is a rank-2 `mdspan` whose layout is `layout_left_padded<padding_stride>`, then `submdspan(A, tuple{r1, r2}, tuple{c1, c2})` also has layout `layout_left_padded<padding_stride>` with the same padding stride as before. The BLAS and algorithms that use it (such as the blocked algorithms in LAPACK) depend on this ability to operate on contiguous submatrices with the same layout as their parent. For this reason, we can replace the `layout_blas_general` layout in P1673 with `layout_left_padded` and `layout_right_padded`. Making most effective use of the new layouts in code that uses P1673 calls for integrating them with `submdspan`. This is why we propose the following changes as well.

§ 3.2.3 Design change from R0 to R1

A design change from R0 to R1 of this paper makes this overalignment case easier to use and more like the existing `std::assume_aligned` interface. In R0 of this paper, the user’s padding input parameter (either a compile-time `padding_stride` or a run-time value) was exactly the padding stride. As such, it had to be greater than or equal to the extent to pad. For example, if users had an extent(0) of 13 and wanted to overalign the corresponding stride(1) to a multiple of 4, they would have had to specify `layout_left_padded<16>`. This was inconsistent with `std::assume_aligned`, whose template argument (the byte alignment) would need to be `4 * sizeof(element_type)`. Also, users who wanted a compile-time padding stride would have needed to compute it themselves from the corresponding compile-time extent, rather than prespecifying a fixed overalignment factor that could be used for any extent. This was not only harder to use, but it made the layout itself (not just the layout mapping) depend on the extent. That was inconsistent with the existing `mdspan` layouts, where the layout type itself (e.g., `layout_left`) is always a function from extents specialization to layout mapping.

In R1 and subsequent revisions of this paper, we interpret the case where the input padding stride is less than the extent to pad as an “overalignment factor” instead of a stride. To revisit the above example, `layout_left_padded<4>` would take an extent(0) of 13 and round up the corresponding stride(1) to 16. However, as before, `layout_left_padded<17>` would take an extent(0) of 13 and round up the corresponding stride(1) to 17. The rule is consistent: the actual padding stride is always the next multiple of the input padding stride greater than or equal to the extent-to-pad.

In R0 of this paper, the following alias

```
using overaligned_matrix_t =  
    mdspan<float, dextents<size_t, 2>, layout_right_padded<4>>;
```

would only be meaningful if the run-time extents are less than or equal to 4. In R1 and subsequent revisions, this alias would always mean “the padding stride rounds up the

rightmost extent to a multiple of 4, whatever the extent may be.” R0 had no way to express that use case with a compile-time input padding stride. This is important for hardware features and compiler optimizations that require overalignment of multidimensional arrays.

§ 3.2.4 Padding stride equality for layout mapping conversions

`layout_left_padded<padding_stride>::mapping<Extents>` has a converting constructor from `layout_left_padded<other_padding_stride>::mapping<OtherExtents>`. Similarly, `layout_right_padded<padding_stride>::mapping<Extents>` has a converting constructor from `layout_right_padded<other_padding_stride>::mapping<OtherExtents>`. These constructors require, among other conditions, that if `padding_stride` and `other_padding_stride` do not equal `dynamic_extent`, then `padding_stride` equals `other_padding_stride`.

Users may ask why they can’t convert a more overaligned mapping, such as `layout_left_padded<4>::mapping`, to a less overaligned mapping, such as `layout_left_padded<2>::mapping`. The problem is that this may not be correct for all extents. For example, the following code would be incorrect if it were well formed (it is not, in this proposal).

```
layout_left_padded<4>::mapping m_orig{extents{9, 2}};  
layout_left_padded<2>::mapping m_new(m_orig);
```

The issue is that `m_orig` has an underlying (“physical”) layout of `extents{12, 2}`, but `layout_left_padded<2>::mapping{extents{9, 2}}` would have an underlying layout of `extents{10, 2}`. That is, `layout_left_padded<4>::mapping{extents{9, 2}}.stride(1)` is 12, but `layout_left_padded<2>::mapping{extents{9, 2}}.stride(1)` is 10.

In case one is tempted to permit assigning dynamic padding stride to static padding stride, the following code would also be incorrect if it were well formed (it is not, in this proposal). Again, `m_orig.stride(1)` is 12.

```
layout_left_padded<dynamic_extent>::mapping m_orig{extents{9, 2}, 4};  
layout_left_padded<2>::mapping m_new(m_orig);
```

The following code is well formed in this proposal, and it gives `m_new` the expected original padding stride of 12.

```
layout_left_padded<dynamic_extent>::mapping m_orig{extents{9, 2}, 4};  
layout_left_padded<dynamic_extent>::mapping m_new(m_orig);
```

Similarly, the following code is well formed in this proposal, and it gives `m_new` the expected original padding stride of 12.

```
layout_left_padded<4>::mapping m_orig{extents{9, 2}};  
layout_left_padded<dynamic_extent>::mapping m_new(m_orig);
```

§ 3.2.5 New layout mapping constructors in R2

R2 of this proposal adds new constructors to `layout_{left,right}_padded::mapping`. First, it adds default constructors that default-construct the `extents_type` object, but otherwise

behave like the `mapping(const extents_type&)` constructor. That is, they fill in the correct run-time padding stride value, if this is possible given the `padding_stride` template argument. Second, R2 adds more converting constructors. For `layout_left_padded::mapping`, R2 adds a converting constructor from each of the following.

- `layout_left::mapping<OtherExtents>`
- `layout_stride::mapping<OtherExtents>`

For `layout_right_padded::mapping`, R2 adds a converting constructor from each of the following.

- `layout_right::mapping<OtherExtents>`
- `layout_stride::mapping<OtherExtents>`

§ 3.2.6 Conversion from `layout_left` to `layout_left_padded`

The converting constructor from `layout_left::mapping` to `layout_left_padded::mapping` exists by analogy with the existing constructor `layout_stride::mapping(const StridedLayoutMapping& other) ([mdspan.layout.stride.cons])` that can convert from `layout_left::mapping` to `layout_stride::mapping`. `layout_left` expresses a special case of `layout_left_padded`, just as `layout_left` expresses a special case of `layout_stride`. Thus, this is an implicit conversion as long as the conversion from the input's `extents_type` to the result's `extents_type` would be implicit.

This conversion is useful for C++ wrappers for the BLAS or LAPACK.

`layout_left_padded<dynamic_extent>::mapping<dextent<int, 2>>` expresses in C++ exactly the 2-D array layout that the BLAS and LAPACK accept, including their requirement that the extents and `stride(1)` all be run-time values. Thus, a C++ wrapper for the BLAS (see P1673) or LAPACK might reasonably have a specialization for `mdspan` with layout `layout_left_padded<dynamic_extent>::mapping<dextent<int, 2>>`, that can call with very few error checks or layout conversions directly into an existing C or Fortran BLAS or LAPACK library. However, users would reasonably want to create their 2-D arrays as `layout_left`, since it's a simpler layout that doesn't need to store the column stride. The converting constructor from `layout_left::mapping` to `layout_left_padded::mapping` would let users or libraries easily convert from the less general `layout_left` to the slightly more general `layout_left_padded` that a C++ BLAS or LAPACK wrapper would naturally use.

§ 3.2.7 Conversion from `layout_stride` to `layout_left_padded`

The converting constructor from `layout_stride::mapping` to `layout_left_padded::mapping` exists by analogy with the existing converting constructor from `layout_stride::mapping` to `layout_left::mapping`. This constructor is explicit for `rank() > 0`, because it always converts from a more general case to a more specific case.

Explicit conversions to `layout_stride::mapping` are useful because `layout_stride::mapping` can express all the layout mappings in the Standard and this

proposal. It’s like a “type-erased” version of all of them. For example, a library of `mdspan` algorithms might reasonably convert to `layout_stride::mapping` for some less performance-critical algorithms, as a way to minimize algorithm instantiations for different layouts.

§ 3.3 Integration with `submdspan`

We propose changing `submdspan` (see P2630) of a `layout_left` resp. `layout_right` `mdspan` to return `layout_left_padded` resp. `layout_right_padded` instead of `layout_stride`, if the slice arguments permit it. Taking the `submdspan` of a `layout_left_padded` resp. `layout_right_padded` `mdspan` will preserve the layout, again if the slice arguments permit it.

The phrase “if the slice arguments permit it” means the following.

§ 3.3.1 `layout_left_padded` and `layout_left` cases

In what follows, let `left_submatrix` be the following function,

```
template<class Elt, class Extents, class Layout,
        class Accessor, class S0, class S1>
requires(
    is_convertible_v<S0,
        tuple<typename Extents::index_type,
            typename Extents::index_type>> and
    is_convertible_v<S1,
        tuple<typename Extents::index_type,
            typename Extents::index_type>>
)
auto left_submatrix(
    mdspan<Elt, Extents, Layout, Accessor> X, S0 s0, S1 s1)
{
    auto full_extents =
        [<size_t ... Indices>(index_sequence<Indices...>) {
            return tuple{ (Indices, full_extent)... };
        } (make_index_sequence<X.rank() - 2>());
    return apply([&](full_extent_t ... fe) {
        return submdspan(X, s0, s1, fe...);
    }, full_extents);
}
```

let `index_type` be an integral type, let `s0` be an object of a type `S0` such that `is_convertible_v<S0, tuple<index_type, index_type>>` is true, and let `s1` be an object of a type `S1` such that `is_convertible_v<S1, tuple<index_type, index_type>>` is true.

Let `X` be an `mdspan` with rank at least two with `decltype(X)::index_type` naming the same type as `index_type`, whose layout is `layout_left_padded<padding_stride_X>` for some `constexpr size_t padding_stride_X`. Let `X_sub` be the object returned from `left_submatrix(X, s0, s1)`. Then, `X_sub` is an `mdspan` of rank `X.rank()` with layout `layout_left_padded<padding_stride_X>`, and `X_sub.stride(1)` equals `X.stride(1)`.

Let `Z` be an `mdspan` with rank at least two with `decltype(Z)::index_type` naming the same type as `index_type`, whose layout is `layout_left`. Let `Z_sub` be the object returned from

left_submatrix(Z, s0, s1). Then, Z_sub is an mdspan of rank Z.rank() with layout layout_left_padded<padding_stride_Z>, where padding_stride_Z is

- srm1_val1 - srm1_val0, if srm1 is convertible to
 tuple<integral_constant<decltype(W)::index_type, srm1_val0>,
 integral_constant<decltype(W)::index_type, srm1_val1>> with srm1_val1
 greater than to equal to srm1_val0; else,
- dynamic_rank.

Also, Z_sub.stride(1) equals Z.stride(1).

§ 3.3.2 layout_right_padded and layout_right cases

In what follows, let right_submatrix be the following function,

```
template<class Elt, class Extents, class Layout,
        class Accessor, class Srm2, class Srm1>
requires(
    is_convertible_v<Srm2,
        tuple<typename Extents::index_type,
            typename Extents::index_type>> and
    is_convertible_v<Srm1,
        tuple<typename Extents::index_type,
            typename Extents::index_type>>
)
auto right_submatrix(
    mdspan<Elt, Extents, Layout, Accessor> X, Srm2 srm2, Srm1 srm1)
{
    auto full_extents =
        [<size_t ... Indices>(index_sequence<Indices...>) {
            return tuple{ (Indices, full_extent)... };
        } (make_index_sequence<X.rank() - 2>());
    return apply([&](full_extent_t ... fe) {
        return submdspan(X, fe..., srm2, srm1);
    }, full_extents);
}
```

let srm2 (“s of rank minus 2”) be an object of a type Srm2 such that is_convertible_v<S0, tuple<index_type_X, index_type_X>> is true, and let srm1 (“s of rank minus 1”) be an object of a type Srm1 such that is_convertible_v<S1, tuple<index_type_X, index_type_X>> is true.

Similarly, let Y be an mdspan with rank at least two whose layout is layout_right_padded<padding_stride_Y> for some constexpr size_t padding_stride_Y. Let index_type_Y name the type decltype(Y)::index_type. Let srm2 (“S of rank minus 2”) be an object of a type Srm2 such that is_convertible_v<Srm2, tuple<index_type_Y, index_type_Y>> is true, and let srm1 (“S of rank minus 1”) be an object of a type Srm1 such that is_convertible_v<Srm1, tuple<index_type_Y, index_type_Y>> is true. In the following code fragment,

```
auto full_extents =
    [<size_t ... Indices>(index_sequence<Indices...>) {
```

```

        return tuple{(Indices, full_extent)...};
    }(make_index_sequence<Y.rank() - 2>());

auto Y_sub = apply([&](full_extent_t... fe) {
    return submdspan(Y, fe..., srm2, srm1);
}, full_extents);

```

Y_sub is an mdspan of rank Y.rank() with layout layout_left_padded<padding_stride>, and Y_sub.stride(1) equals Y.stride(1).

Let Z be an mdspan with rank at least two whose layout is layout_left. Let index_type_Z name the type decltype(Z)::index_type. Let s0 be an object of a type S0 such that is_convertible_v<S0, tuple<index_type_Z, index_type_Z>> is true, and let s1 be an object of a type S1 such that is_convertible_v<S1, tuple<index_type_Z, index_type_Z>> is true. In the following code fragment,

```

auto full_extents =
    [<size_t ... Indices>(index_sequence<Indices...>)] {
        return tuple{(Indices, full_extent)...};
    }(make_index_sequence<Z.rank() - 2>());

auto Z_sub = apply( [&](full_extent_t... fe) {
    return submdspan(Z, s0, s1, fe...);
}, full_extents );

```

Z_sub is an mdspan of rank Z.rank() with layout layout_left_padded<padding_stride_Z>, where padding_stride_Z is s0_val1 - s0_val0 if s0 is convertible to tuple<integral_constant<index_type_Z, s0_val0>, integral_constant<index_type_Z, s0_val1>> with s0_val1 greater than to equal to s0_val0. Also, Z_sub.stride(1) equals Z.stride(1).

Similarly, let w be an mdspan with rank at least two whose layout is layout_right. Let index_type_W name the type decltype(W)::index_type. Let srm2 (“S of rank minus 2”) be an object of a type Srm2 such that is_convertible_v<Srm2, tuple<index_type_W, index_type_W>> is true, and let srm1 (“S of rank minus 1”) be an object of a type Srm1 such that is_convertible_v<Srm1, tuple<index_type_W, index_type_W>> is true. In the following code fragment,

```

auto full_extents =
    [<size_t ... Indices>(index_sequence<Indices...>)] {
        return tuple{(Indices, full_extent)...};
    }(make_index_sequence<W.rank() - 2>());

auto W_sub = apply( [&](full_extent_t... fe) {
    return submdspan(W, fe..., srm2, srm1);
}, full_extents);

```

W_sub is an mdspan of rank W.rank() with layout layout_left_padded<padding_stride_W>, where padding_stride_W is srm1_val1 - srm1_val0 if srm1 is convertible to tuple<integral_constant<index_type_W, srm1_val0>, integral_constant<index_type_W, srm1_val1>> with srm1_val1 greater than to equal to srm1_val0. Also, W_sub.stride(1) equals W.stride(1).

Preservation of these layouts under `submdspan` is an important feature for our linear algebra library proposal P1673, because it means that for existing BLAS and LAPACK use cases, if we start with one of these layouts, we know that we can implement fast linear algebra algorithms by calling directly into an optimized C or Fortran BLAS.

§ 3.4 Examples

§ 3.4.1 Directly call C BLAS without checks

We show examples before and after this proposal of functions that compute the matrix-matrix product $C + = AB$. The `recursive_matrix_product` function computes this product recursively, by partitioning each of the three matrices into a 2×2 block matrix using the `partition` function. When the C matrix is small enough, `recursive_matrix_product` stops recursing and instead calls a `base_case_matrix_product` function with different overloads for different matrix layouts. If the matrix layouts support it, `base_case_matrix_product` can call the C BLAS function `cblas_sgemv` directly on the `mdspan`'s data. This is fast if the C BLAS is optimized. Otherwise, `base_case_matrix_product` falls back to a slow generic implementation.

This example is far from ideally optimized, but it hints at the kind of optimizations that linear algebra computations do in practice.

Common code:

```
template<class Layout>
using out_matrix_view = mdspan<float, dextents<int, 2>, Layout>;

template<class Layout>
using in_matrix_view = mdspan<const float, dextents<int, 2>, Layout>;

// Before this proposal, if Layout is layout_left or layout_right,
// the returned mdspan would all be layout_stride.
// After this proposal, the returned mdspan would be
// layout_left_padded resp. layout_right_padded.
template<class ElementType, class Layout>
auto partition(mdspan<ElementType, dextents<int, 2>, Layout> A)
{
    auto M = A.extent(0);
    auto N = A.extent(1);
    auto A00 = submdspan(A, tuple{0, M / 2}, tuple{0, N / 2});
    auto A01 = submdspan(A, tuple{0, M / 2}, tuple{N / 2, N});
    auto A10 = submdspan(A, tuple{M / 2, M}, tuple{0, N / 2});
    auto A11 = submdspan(A, tuple{M / 2, M}, tuple{N / 2, N});
    return tuple{
        A00, A01,
        A10, A11
    };
}

template<class Layout>
void recursive_matrix_product(in_matrix_view<Layout> A,
    in_matrix_view<Layout> B, out_matrix_view<Layout> C)
{

```

```

// Some hardware-dependent constant
constexpr int recursion_threshold = 16;
if(std::max(C.extent(0) || C.extent(1)) <= recursion_threshold) {
    base_case_matrix_product(A, B, C);
} else {
    auto [C00, C01,
          C10, C11] = partition(C);
    auto [A00, A01,
          A10, A11] = partition(A);
    auto [B00, B01,
          B10, B11] = partition(B);
    recursive_matrix_product(A00, B00, C00);
    recursive_matrix_product(A01, B10, C00);
    recursive_matrix_product(A10, B00, C10);
    recursive_matrix_product(A11, B10, C10);
    recursive_matrix_product(A00, B01, C01);
    recursive_matrix_product(A01, B11, C01);
    recursive_matrix_product(A10, B01, C11);
    recursive_matrix_product(A11, B11, C11);
}
}

// Slow generic implementation
template<class Layout>
void base_case_matrix_product(in_matrix_view<Layout> A,
    in_matrix_view<Layout> B, out_matrix_view<Layout> C)
{
    for(size_t j = 0; j < C.extent(1); ++j) {
        for(size_t i = 0; i < C.extent(0); ++i) {
            typename out_matrix_view<Layout>::value_type C_ij{};
            for(size_t k = 0; k < A.extent(1); ++k) {
                C_ij += A(i,k) * B(k,j);
            }
            C(i,j) += C_ij;
        }
    }
}

```

A user might interpret `layout_left` as “column major,” and therefore “the natural layout to pass into the BLAS.”

```

void base_case_matrix_product(in_matrix_view<layout_left> A,
    in_matrix_view<layout_left> B, out_matrix_view<layout_left> C)
{
    cblas_sgemm(CblasColMajor, CblasNoTrans, CblasNoTrans,
        C.extent(0), C.extent(1), A.extent(1), 1.0f,
        A.data_handle(), A.stride(1), B.data_handle(), B.stride(1),
        1.0f, C.data_handle(), C.stride(1));
}

```

However, `recursive_matrix_product` never gets to use the `layout_left` overload of `base_case_matrix_product`, because the base case matrices are always `layout_stride`.

On discovering this, the author of these functions might be tempted to write a custom layout for “BLAS-compatible” matrices. However, the current version of the submdspan proposal [P2630R2](#) forces `partition` to return four `layout_stride` mdspan if given a `layout_left` (or

layout_right) input mdspan. This would, in turn, force users of recursive_matrix_product to commit to a custom layout, if they want to use the BLAS.

Alternately, the author of these functions could specialize base_case_matrix_product for layout_stride, and check whether A.stride(0), B.stride(0), and C.stride(0) are all equal to one before calling cblas_sgemm. However, that would force extra run-time checks for a use case that most users might never encounter, because most users are starting with layout_left matrices or contiguous submatrices thereof.

After our proposal, the author can specialize base_case_matrix_product for exactly the layout supported by the BLAS. They could even get rid of the fall-back implementation if users never exercise it.

```
template<size_t p>
void base_case_matrix_product(in_matrix_view<layout_left_padded<p>> A,
    in_matrix_view<layout_left_padded<p>> B,
    out_matrix_view<layout_left_padded<p>> C)
{ // same code as above
    cblas_sgemm(CblasColMajor, CblasNoTrans, CblasNoTrans,
        C.extent(0), C.extent(1), A.extent(1), 1.0f,
        A.data_handle(), A.stride(1), B.data_handle(), B.stride(1),
        1.0f, C.data_handle(), C.stride(1));
}
```

§ 3.4.2 Overaligned access

By combining these new layouts with an accessor that ensures overaligned access, we can create an mdspan for which the beginning of every contiguous segment of elements is overaligned by some given factor. This can enable use of hardware features that require overaligned memory access.

The following aligned_accessor class template (which this proposal does *not* propose to add to the C++ Standard Library) uses the C++ Standard Library function assume_aligned to decorate pointer access.

```
template<class ElementType, std::size_t byte_alignment>
struct aligned_accessor {
    // Even if a pointer p is aligned, p + i might not be.
    using offset_policy = std::default_accessor<ElementType>;

    using element_type = ElementType;
    using reference = ElementType&;
    // Some implementations might have an easier time optimizing
    // if this class applies an attribute to the pointer type.
    // Examples of attributes include
    // __declspec(align_value(byte_alignment))
    // and
    // __attribute__((align_value(byte_alignment))).
    using data_handle_type = ElementType*;

    constexpr aligned_accessor() noexcept = default;

    // A feature of default_accessor that permits
    // conversion from nonconst to const.

```

```

template<class OtherElementType, std::size_t other_byte_alignment>
requires (
    std::is_convertible_v<OtherElementType(*)[], element_type(*)[]> &&
    other_byte_alignment == byte_alignment)
constexpr aligned_accessor(
    aligned_accessor<OtherElementType, other_byte_alignment>) noexcept
{}

constexpr reference
access(data_handle_type p, size_t i) const noexcept {
    return std::assume_aligned< byte_alignment >(p)[i];
}

constexpr typename offset_policy::data_handle_type
offset(data_handle_type p, size_t i) const noexcept {
    return p + i;
}
};

```

We include some helper functions for making overaligned array allocations.

```

template<class ElementType>
struct delete_raw {
    void operator()(ElementType* p) const {
        std::free(p);
    }
};

template<class ElementType>
using allocation_t =
    std::unique_ptr<ElementType[], delete_raw<ElementType>>;

template<class ElementType, std::size_t byte_alignment>
allocation_t<ElementType>
allocate_raw(const std::size_t num_elements)
{
    const std::size_t num_bytes = num_elements * sizeof(ElementType);
    void* ptr = std::aligned_alloc(byte_alignment, num_bytes);
    return {ptr, delete_raw<ElementType>{}};
}

```

Now we can show our example. This 15 x 17 matrix of float will have extra padding so that every column is aligned to $8 * \text{sizeof(float)}$ bytes. We can use the layout mapping to determine the required storage size (including padding). Users can then prove at compile time that they can use special hardware features that require overaligned access and/or assume that the padding element at the end of each column is accessible memory.

```

constexpr std::size_t element_alignment = 8;
constexpr std::size_t byte_alignment =
    element_alignment * sizeof(float);

using layout_type = layout_left_padded<element_alignment>;
layout_type::mapping mapping{dextents<int, 2>{15, 17}};
auto allocation =
    allocate_raw<float, byte_alignment>(mapping.required_span_size());

```

```
using accessor_type = aligned_accessor<float, byte_alignment>;
mdspan m{allocation.get(), mapping, accessor_type{}};

// m_sub has the same layout as m,
// and each column of m_sub has the same overalignment.
auto m_sub = submdspan(m, tuple{0, 11}, tuple{1, 13});
```

§ 3.5 Alternatives

We considered a variant of `layout_stride` that could encode any combination of compile-time or run-time strides in the layout type. This could, for example, use the same mechanism that `extents` uses. (The reference implementation calls this mechanism a “partially static array.”) However, we rejected this approach as overly complex for our design goals.

First, the goal of `layout_{left,right}_padded` isn’t to insist even harder that the compiler bake constants into `mapping::operator()` evaluation. The goal is to communicate compile-time information to *users*. The most benefit comes not just from knowing the padding stride at compile time, but also from knowing that one dimension always uses stride-one (contiguous) storage. Putting these two pieces of information together lets users apply compiler annotations like `assume_aligned`, as in the above `aligned_accessor` example. Knowing that one dimension always uses contiguous storage also tells users that they can pass the `mdspan`’s data directly into C or Fortran libraries like the BLAS or LAPACK. Users can benefit from this even if the padding stride is a run-time value.

Second, the `constexpr` annotations in the existing layout mappings mean that users might be evaluating `layout_stride::mapping::operator()` fully at compile time. The reference `mdspan` implementation has [several tests](#) that demonstrate this by using the result of a layout mapping evaluation in a context where it needs to be known at compile time.

Third, the performance benefit of storing some strides as compile-time constants goes down as the rank increases, because most of the strides would end up depending on run-time values anyway. Strided `mdspan` generally come from a subview of an existing `layout_left` or `layout_right` `mdspan`. In that case, the representation of the strides that preserves the most compile-time information would be just the original `mdspan`’s `extents_type` object. (Compare to the exposition-only *inner-mapping* which we use in the wording for `layout_{left,right}_padded`.) Computing each stride would then call for a forward (for `layout_left`) or reverse (for `layout_right`) product of the original `mdspan`’s extents. As a result, any stride to the right resp. left of a run-time extent would end up depending on that run-time extent anyway. The larger the rank, the more strides get “touched” by run-time information.

Fourth, a strided `mdspan` that can represent layouts as general as `layout_stride`, but has entirely compile-time extents *and* strides, could be useful for supporting features of a specific computer architecture. However, these hardware features would probably have limitations that would prevent them from supporting general strided layouts anyway. For example, they might require strides to be a power of two, or they might be limited to specific ranges of extents or strides. These limitations would call for custom implementation-specific layouts, not something as general as a “compile-time `layout_stride`.”

§ 3.6 Implementation experience

Pull request [180](#) in the [reference mdspan implementation](#) implements most of this proposal. Next steps are to add constructors to the existing layout mappings, and to add submdspan support for the new layouts.

§ 3.7 Desired ship vehicle

C++26 / IS.

§ 4 Wording

Text in blockquotes is not proposed wording, but rather instructions for generating proposed wording. The  character is used to denote a placeholder section number which the editor shall determine. First, apply all wording from P2630R2. (This proposal is a “rebase” atop the changes proposed by P2630R2.)

Add the following feature test macro to *[version.syn]*, replacing `YYYYMML` with the integer literal encoding the appropriate year (YYYY) and month (MM).

```
#define __cpp_lib_mdspan_layout_padded YYYYMML // also in <mdspan>
```

In Section  *[mdspan.syn]*, in the synopsis, after `struct layout_stride;`, add the following:

```
template<size_t padding_stride = dynamic_extent>
struct layout_left_padded {
    template<class Extents>
    class mapping;
};
template<size_t padding_stride = dynamic_extent>
struct layout_right_padded {
    template<class Extents>
    class mapping;
};
```

After paragraph 1 of *[mdspan.layout.policy.overview]*, add the following paragraph 2:

- Each specialization of `layout_left_padded` and `layout_right_padded` meets the layout mapping policy requirements and is a trivial type.

In Section  *[mdspan.layout.left.overview]* (“Overview”), add the following constructor to the `layout_left::mapping` class declaration, between the constructor converting from `layout_right::mapping<OtherExtents>` and the constructor converting from `layout_stride::mapping<OtherExtents>`:

```
template<size_t other_padding_stride, class OtherExtents>
constexpr explicit(! is_convertible_v<OtherExtents, extents_type>)
```



```
mapping(const
    layout_left_padded<other_padding_stride>::mapping<OtherExtents>&)
    noexcept;
```

In Section [◆ \[mdspan.layout.left.cons\]](#) (“Constructors”), add the following between the constructor converting from `layout_right::mapping<OtherExtents>` (ending paragraph 8) and the constructor converting from `layout_stride::mapping<OtherExtents>` (starting paragraph 9 before this proposal), then renumber the following paragraphs in that section accordingly.

```
template<size_t other_padding_stride, class OtherExtents>
constexpr explicit(! is_convertible_v<OtherExtents, extents_type>)
    mapping(const
        layout_left_padded<other_padding_stride>::mapping<OtherExtents>&
        other) noexcept;
```

9 *Constraints:* `is_constructible_v<extents_type, OtherExtents>` is true.

10 *Mandates:* If

- `Extents::rank()` is greater than one,
- `Extents::static_extent(0)` does not equal `dynamic_extent`,
- `OtherExtents::static_extent(0)` does not equal `dynamic_extent`, and
- `other_padding_stride` does not equal `dynamic_extent`,

then `Extents::static_extent(0)` is a multiple of `other_padding_stride`.

11 *Preconditions:*

- if `extents_type::rank() > 1` is true, then `other.stride(1)` equals `other.extents(0)`, and
- `other.required_span_size()` is representable as a value of type `index_type` (*[basic.fundamental]*).

12 *Effects:* Direct-non-list-initializes `extents_` with `other.extents()`.

In Section [◆ \[mdspan.layout.right.overview\]](#) (“Overview”), add the following constructor to the `layout_right::mapping` class declaration, between the constructor converting from `layout_left::mapping<OtherExtents>` and the constructor converting from `layout_stride::mapping<OtherExtents>`.

```
template<size_t other_padding_stride, class OtherExtents>
constexpr explicit(! is_convertible_v<OtherExtents, extents_type>)
    mapping(const
        layout_right_padded<other_padding_stride>::mapping<OtherExtents>&)
    noexcept;
```

In Section [◆ \[mdspan.layout.right.cons\]](#) (“Constructors”), add the following between the constructor converting from

layout_left::mapping<OtherExtents> (ending paragraph 8) and the constructor converting from layout_stride::mapping<OtherExtents> (starting paragraph 9 before this proposal), then renumber the following paragraphs in that section accordingly.

```
template<size_t other_padding_stride, class OtherExtents>
constexpr explicit(! is_convertible_v<OtherExtents, extents_type>)
    mapping(const
        layout_right_padded<other_padding_stride>::mapping<OtherExtents>&
        other) noexcept;
```

9 *Constraints:* is_constructible_v<extents_type, OtherExtents> is true.

10 *Mandates:* If

- Extents::rank() is greater than one,
- Extents::static_extent(Extents::rank() - 1) does not equal dynamic_extent,
- OtherExtents::static_extent(Extents::rank() - 1) does not equal dynamic_extent, and
- other_padding_stride does not equal dynamic_extent,

then Extents::static_extent(Extents::rank() - 1) is a multiple of other_padding_stride.

11 *Preconditions:*

- if extents_type::rank() > 1 is true, then other.stride(extents_type::rank() - 2) equals other.extents().extent(extents_type::rank() - 1).
- other.required_span_size() is representable as a value of type index_type ([basic.fundamental]).

12 *Effects:* Direct-non-list-initializes extents_ with other.extents().

In Section  [mdspan.layout.stride.cons], in paragraph 7 (Remarks for the constructor layout_stride::mapping(const StridedLayoutMapping&)), right after the word Remarks, add the following text.

Let *is-layout-left-padded-mapping-of* be the exposition-only variable template defined as follows.

```
template<class Layout>
struct is-layout-left-padded : // exposition only
    false_type {};

template<size_t padding_stride>
struct is-layout-left-padded<layout_left_padded<padding_stride>> : //
    exposition only
    true_type {};

template<class Mapping>
```

```
constexpr bool is-layout-left-padded-mapping-of // exposition only
is-layout-left-padded<typename Mapping::layout_type>::value;
```

Let *is-layout-right-padded-mapping-of* be the exposition-only variable template defined as follows.

```
template<class Layout>
struct is-layout-left-padded : // exposition only
    false_type {};

template<size_t padding_stride>
struct is-layout-left-padded<layout_left_padded<padding_stride>> : //
    exposition only
    true_type {};

template<class Mapping>
constexpr bool is-layout-left-padded-mapping-of // exposition only
is-layout-left-padded<typename Mapping::layout_type>::value;
```

In Section [◆ \[mdspan.layout.stride.cons\]](#), in paragraph 7 (Remarks for the constructor `layout_stride::mapping(const StridedLayoutMapping&)`), add the following two lines immediately below *is-mapping-of*<layout_right, LayoutStrideMapping> || and above *is-mapping-of*<layout_stride, LayoutStrideMapping> ||:

```
is-layout-left-padded-mapping-of <LayoutStrideMapping> ||
is-layout-right-padded-mapping-of <LayoutStrideMapping> ||
```

After the end of Section [◆ \[mdspan.layout.stride\]](#), add the following:

§ 4.1 Class template `layout_left_padded::mapping` [mdspan.layout.left_padded]

- 1 `layout_left_padded` provides a layout mapping that behaves like `layout_left::mapping`, except that the *padding stride* `stride(1)` can be greater than or equal to `extent(0)`. Users provide an input padding stride value either as a `size_t` template parameter `padding_stride` of `layout_left_padded`, or as a run-time argument of `layout_left_padded::mapping`'s constructor. The padding stride is the least multiple of the input padding stride value greater than or equal to `extent(0)`.

```
template<size_t padding_stride>
template<class Extents>
class layout_left_padded<padding_stride>::mapping {
public:
    using extents_type = Extents;
    using index_type = typename extents_type::index_type;
    using size_type = typename extents_type::size_type;
    using rank_type = typename extents_type::rank_type;
    using layout_type = layout_left_padded<padding_stride>;

private:
    static constexpr size_t actual-padding-stride = /* see-below */; //
        exposition only
```

```

using inner-extents-type = /* see-below */; // exposition only
using unpadded-extent-type = /* see-below */; // exposition only
using inner-mapping-type =
    layout_left::template mapping<inner-extents-type>; // exposition only

inner-mapping-type inner-mapping; // exposition only
unpadded-extent-type unpadded-extent; // exposition only

```

```

public:
    constexpr mapping()
        requires(actual-padding-stride != dynamic_extent) noexcept = default;
    constexpr mapping()
        requires(actual-padding-stride == dynamic_extent) noexcept;

    constexpr mapping(const mapping&) noexcept = default;
    mapping& operator=(const mapping&) noexcept = default;

    constexpr mapping(const extents_type& ext);

    template<class Size>
        constexpr mapping(const extents_type& ext, Size padding_value);

    template<class OtherExtents>
        constexpr explicit(! is_convertible_v<OtherExtents, extents_type>)
            mapping(const layout_left::mapping<OtherExtents>&);

    template<class OtherExtents>
        constexpr explicit(extents_type::rank() > 0)
            mapping(const layout_stride::mapping<OtherExtents>&);

    template<size_t other_padding_stride, class OtherExtents>
        constexpr explicit( /* see below */ )
            mapping(const
                layout_left_padded<other_padding_stride>::mapping<OtherExtents>&);

    template<size_t other_padding_stride, class OtherExtents>
        constexpr explicit(! is_convertible_v<OtherExtents, extents_type>)
            mapping(const
                layout_right_padded<other_padding_stride>::mapping<OtherExtents>&)
                noexcept;

    constexpr extents_type extents() const noexcept;

    constexpr std::array<index_type, extents_type::rank()>
        strides() const noexcept;

    constexpr index_type required_span_size() const noexcept;

    template<class... Indices>
        constexpr index_type operator()(Indices... idxs) const noexcept;

    static constexpr bool is_always_unique() noexcept { return true; }
    static constexpr bool is_always_exhaustive() noexcept;
    static constexpr bool is_always_strided() noexcept { return true; }

    static constexpr bool is_unique() noexcept { return true; }
    constexpr bool is_exhaustive() const noexcept;
    static constexpr bool is_strided() noexcept { return true; }

```

```
constexpr index_type stride(rank_type r) const noexcept;

template<size_t other_padding_stride, class OtherExtents>
friend constexpr bool operator==(
    const mapping&,
    const typename
        layout_left_padding<other_padding_stride>::mapping<OtherExtents>&)
    noexcept;
};
```

2 Throughout this section, let `P_left` be the following `size_extents_type::rank()` parameter pack of `size_t`:

- (2.1) — If `extents_type::rank()` equals zero or one, then the empty parameter pack;
- (2.2) — else, the parameter pack `size_t(1), size_t(2), ..., extents_type::rank() - 1`.

3 *Mandates:* If

- `extents_type::rank()` is greater than one,
- `padding_stride` does not equal `dynamic_extent`, and
- `extents_type::static_extent(0)` does not equal `dynamic_extent`,

then the least multiple of `padding_stride` that is greater than or equal to `extents_type::static_extent(0)` is representable as a value of type `size_t`, and is representable as a value of type `index_type`.

```
static constexpr size_t actual-padding-stride = /* see-below */; //
    exposition only
```

- (4.1) — If `extents_type::rank()` equals zero or one, then `padding_stride`.

- (4.2) — Else, if

- `padding_stride` does not equal `dynamic_extent` and
- `extents_type::static_extent(0)` does not equal `dynamic_extent`,

then the `size_t` value which is the least multiple of `padding_stride` that is greater than or equal to `extents_type::static_extent(0)`.

- (4.3) — Otherwise, `dynamic_extent`.

```
using inner-extents-type = /* see-below */; // exposition only
```

- (5.1) — If `extents_type::rank()` equals zero or one, then `inner-extents-type` names the type `extents_type`.

- (5.2) — Otherwise, `inner-extents-type` names the type `extents<index_type, actual-padding-stride, extents_type::static_extent(P_left) ...>`.

```
using unpadded-extent-type = /* see-below */; // exposition only
```

- (6.1) — If `extents_type::rank()` equals zero, then *unpadded-extent-type* names the type `extents<index_type>`.
- (6.2) — Otherwise, *unpadded-extent-type* names the type `extents<index_type, extents_type::static_extent(0)>`.

```
constexpr mapping()
requires(actual-padding-stride == dynamic_extent) noexcept;
```

7 *Effects:* Equivalent to `mapping(extents_type{});`.

```
constexpr mapping(const extents_type& ext);
```

8 *Preconditions:* If `extents_type::rank()` is greater than one and `padding_stride` does not equal `dynamic_extent`, then the least multiple of `padding_stride` greater than to equal to `ext.extent(0)` is representable as a value of type `index_type`.

9 *Effects:*

- (9.1) — Direct-non-list-initializes *inner-mapping* with:
 - `ext`, if `extents_type::rank()` is zero or one; else,
 - `ext.extent(0)`, `ext.extent(P_left)...`, if `padding_stride` is `dynamic_extent`; else,
 - `S_left`, `ext.extent(P_left)...`, where `S_left` is the least multiple of `padding_stride` greater than or equal to `ext.extent(0)`; and
- (9.2) — if `extents_type::rank()` is zero, value-initializes *unpadded-extent*; else, direct-non-list-initializes *unpadded-extent* with `ext.extent(0)`.

```
template<class Size>
constexpr mapping(const extents_type& ext, Size padding_value);
```

10 *Constraints:*

- (10.1) — `is_convertible_v<Size, index_type>` is true, and
- (10.2) — `is_nothrow_constructible_v<index_type, Size>` is true.

11 *Preconditions:*

- (11.1) — If `padding_stride` does not equal `dynamic_extent`, then
 - `padding_value` is representable as a value of type `index_type`, and
 - the result of converting `padding_value` to `index_type` equals `padding_stride`.
- (11.2) — If `extents_type::rank()` is greater than one, then the least multiple of `padding_value` greater than to equal to `ext.extent(0)` is representable as a value of type `index_type`.

12 *Effects:*

- (12.1) — Direct-non-list-initializes *inner-mapping* with:

- `ext`, if `extents_type::rank()` is zero or one; else,
- `S_left`, `ext.extent(P_left)...`, where `S_left` is the least multiple of `padding_value` greater than or equal to `ext.extent(0)`; and

- (12.2) — if `extents_type::rank()` is zero, value-initializes *unpadded-extent*; else, direct-non-list-initializes *unpadded-extent* with `ext.extent(0)`.

```
template<class OtherExtents>
constexpr explicit(! is_convertible_v<OtherExtents, extents_type>)
mapping(const layout_left::mapping<OtherExtents>& other);
```

- 13 *Constraints:* `is_constructible_v<extents_type, OtherExtents>` is true.

- 14 *Mandates:* If `OtherExtents::rank() > 1`, *actual-padding-stride* does not equal `dynamic_extent`, and `OtherExtents::static_extent(0)` does not equal `dynamic_extent`, then *actual-padding-stride* equals `OtherExtents::static_extent(0)`.

- 15 *Preconditions:*

- (15.1) — If `extents_type::rank() > 1` is true and `padding_stride == dynamic_extent` is false, then `other.stride(1)` equals the least multiple of `padding_stride` greater than or equal to `extents_type::index-cast(other.extents().extent(0))`; and

- (15.2) — `other.required_span_size()` is representable as a value of type `index_type` ([*basic.fundamental*]).

- 16 *Effects:*

- (16.1) — Direct-non-list-initializes *inner-mapping* with:

- `other.extents()`, if `extents_type::rank()` is zero or one; else
- `other.stride(1)`, `other.extents().extent(P_left)...`; and

- (16.2) — if `extents_type::rank()` is zero, value-initializes *unpadded-extent*; else, direct-non-list-initializes *unpadded-extent* with `other.extents().extent(0)`.

```
template<class OtherExtents>
constexpr explicit(extents_type::rank() > 0)
mapping(const layout_stride::mapping<OtherExtents>& other);
```

- 17 *Constraints:* `is_constructible_v<extents_type, OtherExtents>` is true.

- 19 *Preconditions:*

- (19.1) — If `extents_type::rank() > 1` is true and `padding_stride == dynamic_extent` is false, then `other.stride(1)` equals the least multiple of `padding_stride` greater than or equal to `extents_type::index-cast(other.extents().extent(0))`; and

- (19.2) — If `extents_type::rank() > 0` is true, then `other.stride(0)` equals 1;

- (19.3) — If `extents_type::rank() > 2` is true, and then for all `r` in the range `[ 2, extents_type::rank() )`, `other.stride(r)` equals `other.extents().fwd-prod-of-`

`extents(r)/ other.extents().extent(0) * other.stride(1).`

- (19.4) — `other.required_span_size()` is representable as a value of type `index_type` ([*basic.fundamental*]).

20 *Effects:*

- (20.1) — Direct-non-list-initializes *inner-mapping* with:

— `other.extents()`, if `extents_type::rank()` is zero or one; else

— `other.stride(1), other.extents().extent(P_left)...`; and

- (20.2) — if `extents_type::rank()` is zero, value-initializes *unpadded-extent*; else, direct-non-list-initializes *unpadded-extent* with `other.extents().extent(0)`.

```
template<size_t other_padding_stride, class OtherExtents>
constexpr explicit( /* see below */ )
mapping(const
    layout_left_padded<other_padding_stride>::mapping<OtherExtents>&
    other);
```

21 *Constraints:* `is_constructible_v<extents_type, OtherExtents>` is true.

22 *Mandates:* `padding_stride == dynamic_extent || other_padding_stride == dynamic_extent || padding_stride == other_padding_stride` is true.

23 *Preconditions:*

- (23.1) — If `extents_type::rank() > 1` is true and `padding_stride` does not equal `dynamic_extent`, then `other.stride(1)` equals the least multiple of `padding_stride` greater than or equal to `extents_type::index-cast(other.extent(0))`; and

- (23.2) — `other.required_span_size()` is representable as a value of type `index_type` ([*basic.fundamental*]).

24 *Effects:*

- (24.1) — Direct-non-list-initializes *inner-mapping* with:

— `other.extents()`, if `extents_type::rank()` is zero or one; else

— `other.stride(1), other.extents().extent(P_left)...`; and

- (24.2) — if `extents_type::rank()` is zero, value-initializes *unpadded-extent*; else, direct-non-list-initializes *unpadded-extent* with `other.extents().extent(0)`.

25 *Remarks:* The expression inside `explicit` is equivalent to: `extents_type::rank() > 1 && (padding_stride == dynamic_extent || other_padding_stride == dynamic_extent)`.

```
template<size_t other_padding_stride, class OtherExtents>
constexpr explicit(! is_convertible_v<OtherExtents, extents_type>)
mapping(const
    layout_right_padded<other_padding_stride>::mapping<OtherExtents>&)
noexcept;
```


26 *Constraints:*

- (26.1) — `extents_type::rank()` equals zero or one,
- (26.2) — `is_constructible_v<extents_type, OtherExtents>` is true.

27 *Precondition:* `other.required_span_size()` is representable as a value of type `index_type` ([*basic.fundamental*]).

28 *Effects:*

- (28.1) — Direct-non-list-initializes *inner-mapping* with `other.extents()`; and
- (28.2) — if `extents_type::rank()` is zero, value-initializes *unpadded-extent*; else, direct-non-list-initializes *unpadded-extent* with `other.extents().extent(0)`.

[*Note:* Neither mapping uses the padding stride in the rank-0 or rank-1 case, so the padding stride does not affect either the constraints or the preconditions. – *end note*]

```
constexpr extents_type extents() const noexcept;
```

29 *Effects:*

- (29.1) — If `extents_type::rank()` is zero, equivalent to return `extents_type{}`;
- (29.2) — Otherwise, equivalent to return `extents_type(unpadded-extent_.extent(0), inner-mapping.extent(P_left)...) ;`.

```
constexpr std::array<index_type, extents_type::rank()>  
strides() const noexcept;
```

30 *Effects:* Equivalent to return `inner-mapping.strides()`;

```
constexpr index_type required_span_size() const noexcept;
```

31 *Effects:* Equivalent to return `inner-mapping.required_span_size()`;

```
template<class... Indices>  
constexpr size_t operator()(Indices... i) const noexcept;
```

32 *Constraints:*

- (32.1) — `sizeof...(Indices) == Extents::rank()` is true,
- (32.2) — `(is_convertible_v<Indices, index_type> && ...)` is true, and
- (32.3) — `(is_nothrow_constructible<index_type, Indices> && ...)` is true.

33 *Precondition:* `extents_type::index-cast(i)` is a multidimensional index in `extents()` ([*mdspan.overview*]).

34 *Effects:* Let P be a parameter pack such that `is_same_v<index_sequence_for<Indices...>, index_sequence<P...>>` is true. Equivalent to: return `((static_cast<index_type>(i) * stride(P)) + ... + 0)`;

[Note: Effects are also equivalent to `return inner-mapping(i...);`, but only after the Precondition has been applied. – end note]

```
static constexpr bool is_always_exhaustive() noexcept;
```

35 *Returns:*

- (35.1) — If `extents_type::rank()` equals zero or one, then true;
- (35.2) — else, if neither `inner-mapping-type::static_extent(0)` nor `extents_type::static_extent(0)` equal `dynamic_extent`, then `inner-mapping-type::static_extent(0) == extents_type::static_extent(0)`;
- (35.3) — otherwise, false.

```
constexpr bool is_exhaustive() const noexcept;
```

36 *Returns:*

- (36.1) — If `extents_type::rank()` equals zero, then true;
- (36.2) — else, `inner-mapping.extent(0) == unpadded-extent.extent(0)`.

```
constexpr index_type stride(rank_type r) const noexcept;
```

37 *Effects:* Equivalent to `return inner-mapping.stride(r);`.

```
template<size_t other_padding_stride, class OtherExtents>
friend constexpr bool operator==(
    const mapping& x,
    const typename
        layout_left_padding<other_padding_stride>::mapping<OtherExtents>& y)
    noexcept;
```

38 *Constraints:* `OtherExtents::rank() == extents_type::rank()` is true.

39 *Returns:* true if

- (39.1) — `x.extents() == y.extents()` is true; and
- (39.2) — if `extents_type::rank() > 1` is true, then `x.stride(1) == y.stride(1)` is true.

§ 4.2 Class template `layout_right_padded::mapping` `[mdspan.layout.right_padded]`

- 1 `layout_right_padded` provides a layout mapping that behaves like `layout_right::mapping`, except that the *padding stride* `stride(rank() - 2)` can be greater than or equal to `extent(rank() - 1)`. Users provide an input padding stride value either as a `size_t` template parameter `padding_stride` of `layout_right_padded`, or as a run-time argument of `layout_right_padded::mapping`'s constructor. The padding stride is the least multiple of the input padding stride value greater than or equal to `extent(rank() - 1)`.

```
template<size_t padding_stride>
template<class Extents>
```

```

class layout_right_padded<padding_stride>::mapping {
public:
    using extents_type = Extents;
    using index_type = typename extents_type::index_type;
    using size_type = typename extents_type::size_type;
    using rank_type = typename extents_type::rank_type;
    using layout_type = layout_right_padded<padding_stride>;

private:
    static constexpr size_t actual-padding-stride = /* see-below */; //
        exposition only

    using inner-extents-type = /* see-below */; // exposition only
    using unpadded-extent-type = /* see-below */; // exposition only
    using inner-mapping-type =
        layout_right::template mapping<inner-extents-type>; // exposition only

    inner-mapping-type inner-mapping; // exposition only
    unpadded-extent-type unpadded-extent; // exposition only

public:
    constexpr mapping()
        requires(actual-padding-stride != dynamic_extent) noexcept = default;
    constexpr mapping()
        requires(actual-padding-stride == dynamic_extent) noexcept;

    constexpr mapping(const mapping&) noexcept = default;
    mapping& operator=(const mapping&) noexcept = default;

    constexpr mapping(const extents_type& ext);

    template<class Size>
    constexpr mapping(const extents_type& ext, Size padding_value);

    template<class OtherExtents>
    constexpr explicit(! is_convertible_v<OtherExtents, extents_type>)
        mapping(const layout_right::mapping<OtherExtents>&);

    template<class OtherExtents>
    constexpr explicit(extents_type::rank() > 0)
        mapping(const layout_stride::mapping<OtherExtents>&);

    template<size_t other_padding_stride, class OtherExtents>
    constexpr explicit( /* see below */ )
        mapping(const
            layout_right_padded<other_padding_stride>::mapping<OtherExtents>&
            other);

    template<size_t other_padding_stride, class OtherExtents>
    constexpr explicit(! is_convertible_v<OtherExtents, extents_type>)
        mapping(const
            layout_left_padded<other_padding_stride>::mapping<OtherExtents>&
            other) noexcept;

    constexpr extents_type extents() const noexcept;

    constexpr std::array<index_type, extents_type::rank()>

```

```

    strides() const noexcept;

constexpr index_type required_span_size() const noexcept;

template<class... Indices>
constexpr index_type operator()(Indices... idxs) const noexcept;

static constexpr bool is_always_unique() noexcept { return true; }
static constexpr bool is_always_exhaustive() noexcept;
static constexpr bool is_always_strided() noexcept { return true; }

static constexpr bool is_unique() noexcept { return true; }
constexpr bool is_exhaustive() const noexcept;
static constexpr bool is_strided() noexcept { return true; }

constexpr index_type stride(rank_type r) const noexcept;

template<size_t other_padding_stride, class OtherExtents>
friend constexpr bool operator==(
    const mapping&,
    const typename
        layout_right_padding<other_padding_stride>::mapping<OtherExtents>&)
    noexcept;
};

```

2 Throughout this section, let `Pright` be the following `size_extents_type::rank()` parameter pack of `size_t`:

- (2.1) — If `extents_type::rank()` equals zero or one, then the empty parameter pack;
- (2.2) — else, the parameter pack `size_t(0)`, `size_t(1)`, ..., `extents_type::rank() - 2`.

3 *Mandates:* If

- (3.1) — `extents_type::rank()` is greater than one,
- (3.2) — `padding_stride` does not equal `dynamic_extent`, and
- (3.3) — `extents_type::static_extent(extents_type::rank() - 1)` does not equal `dynamic_extent`,

then the least multiple of `padding_stride` that is greater than or equal to `extents_type::static_extent(extents_type::rank() - 1)` is representable as a value of type `size_t`, and is representable as a value of type `index_type`.

```

static constexpr size_t actual-padding-stride = /* see-below */; //
    exposition only

```

- (4.1) — If `extents_type::rank()` equals zero or one, then `padding_stride`.
- (4.2) — Else, if
 - `padding_stride` does not equal `dynamic_extent` and
 - `extents_type::static_extent(0)` does not equal `dynamic_extent`,

then the `size_t` value which is the least multiple of `padding_stride` that is greater than or equal to `extents_type::static_extent(0)`.

- (4.3) — Otherwise, `dynamic_extent`.

```
using inner-extents-type = /* see-below */; // exposition only
```

- (5.1) — If `extents_type::rank()` equals zero or one, then `inner-extents-type` names the type `extents_type`.

- (5.2) — Otherwise, `inner-extents-type` names the type `extents<index_type, extents_type::static_extent(P_right)..., actual-padding-stride>`.

```
using unpadded-extent-type = /* see-below */; // exposition only
```

- (6.1) — If `extents_type::rank()` equals zero, then `unpadded-extent-type` names the type `extents<index_type>`.

- (6.2) — Otherwise, `unpadded-extent-type` names the type `extents<index_type, extents_type::static_extent(Extents::rank() - 1)>`.

```
constexpr mapping()  
requires(actual-padding-stride == dynamic_extent) noexcept;
```

- 7 *Effects:* Equivalent to `mapping(extents_type{})`;

```
constexpr mapping(const extents_type& ext);
```

- 8 *Preconditions:* If `extents_type::rank()` is greater than one and `padding_stride` does not equal `dynamic_extent`, then the least multiple of `padding_stride` greater than or equal to `ext.extent(extents_type::rank() - 1)` is representable as a value of type `index_type`.

- 9 *Effects:*

- (9.1) — Direct-non-list-initializes `inner-mapping` with:

- `ext`, if `extents_type::rank()` is zero or one; else,
- `ext.extent(P_right)..., ext.extent(extents_type::rank() - 1)`, if `padding_stride` is `dynamic_extent`; else,
- `ext.extent(P_right)..., S_right`, where `S_right` is the least multiple of `padding_stride` greater than or equal to `ext.extent(extents_type::rank() - 1)`; and

- (9.2) — if `extents_type::rank()` is zero, value-initializes `unpadded-extent`; else, direct-non-list-initializes `unpadded-extent` with `ext.extent(extents_type::rank() - 1)`.

```
template<class Size>  
constexpr mapping(const extents_type& ext, Size padding_value);
```

- 10 *Constraints:*

- (10.1) — `is_convertible_v<Size, index_type>` is true, and

(10.2) — `is_nothrow_constructible_v<index_type, Size>` is true.

11 *Preconditions:*

- (11.1) — If `padding_stride` does not equal `dynamic_extent`, then
- `padding_value` is representable as a value of type `index_type`, and
 - the result of converting `padding_value` to `index_type` equals `padding_stride`.
- (10.2) — If `extents_type::rank()` is greater than one, then the least multiple of `padding_value` greater than or equal to `ext.extent(extents_type::rank() - 1)` is representable as a value of type `index_type`.

12 *Effects:*

- (12.1) — Direct-non-list-initializes *inner-mapping* with:
- `ext`, if `extents_type::rank()` is zero or one; else
 - `ext.extent(P_right)...`, `S_right`, where `S_right` is the least multiple of `padding_value` greater than or equal to `ext.extent(extents_type::rank() - 1)`; and
- (12.2) — if `extents_type::rank()` is zero, value-initializes *unpadded-extent*; else, direct-non-list-initializes *unpadded-extent* with `ext.extent(extents_type::rank() - 1)`.

```
template<class OtherExtents>
constexpr explicit(! is_convertible_v<OtherExtents, extents_type>)
mapping(const layout_right::mapping<OtherExtents>& other);
```

13 *Constraints:* `is_constructible_v<extents_type, OtherExtents>` is true.

14 *Mandates:* If `OtherExtents::rank() > 1`, *actual-padding-stride* does not equal `dynamic_extent`, and `OtherExtents::static_extent(extents_type::rank() - 1)` does not equal `dynamic_extent`, then *actual-padding-stride* equals `OtherExtents::static_extent(extents_type::rank() - 1)`.

15 *Preconditions:*

- (15.1) — If `extents_type::rank() > 1` is true and `padding_stride` does not equal `dynamic_extent`, then `other.stride(extents_type::rank() - 2)` equals the least multiple of `padding_stride` greater than or equal to `extents_type::index-cast(other.extents().extent(extents_type::rank() - 1))`; and
- (15.2) — `other.required_span_size()` is representable as a value of type `index_type` (*[basic.fundamental]*).

16 *Effects:*

- (16.1) — Direct-non-list-initializes *inner-mapping* with:
- `other.extents()`, if `extents_type::rank()` is zero or one; else,

— `other.extents().extent(P_right)...`, `other.stride(extents_type::rank() - 2)`; and

- (16.2) — if `extents_type::rank()` is zero, value-initializes *unpadded-extent*; else, direct-non-list-initializes *unpadded-extent* with `other.extents().extent(extents_type::rank() - 1)`.

```
template<class OtherExtents>
constexpr explicit(extents_type::rank() > 0)
mapping(const layout_stride::mapping<OtherExtents>& other);
```

17 *Constraints:* `is_constructible_v<extents_type, OtherExtents>` is true.

19 *Preconditions:*

- (19.1) — If `extents_type::rank() > 1` is true and `padding_stride == dynamic_extent` is false, then `other.stride(extents_type::rank()-2)` equals the least multiple of `padding_stride` greater than or equal to `extents_type::index-cast(other.extents().extent(extents_type::rank()-1))`; and
- (19.2) — If `extents_type::rank() > 0` is true, then `other.stride(extents_type::rank() - 1)` equals 1; and
- (19.3) — If `extents_type::rank() > 2` is true, then for all `r` in the range `[0, extents_type::rank()-2)`, `other.stride(r)` equals `extents().rev-prod-of-extents(r) / other.extents().extent(extents_type::rank()-1) * other.stride(extents_type::rank()-1)`.
- (19.4) — `other.required_span_size()` is representable as a value of type `index_type` ([*basic.fundamental*]).

20 *Effects:*

- (20.1) — Direct-non-list-initializes *inner-mapping* with:
- `other.extents()`, if `extents_type::rank()` is zero or one; else,
 - `other.extents().extent(P_right)...`, `other.stride(extents_type::rank() - 2)`; and
- (20.2) — if `extents_type::rank()` is zero, value-initializes *unpadded-extent*; else, direct-non-list-initializes *unpadded-extent* with `other.extents().extent(extents_type::rank() - 1)`.

```
template<size_t other_padding_stride, class OtherExtents>
constexpr explicit( /* see below */ )
mapping(const
    layout_right_padded<other_padding_stride>::mapping<OtherExtents>&
    other);
```

21 *Constraints:* `is_constructible_v<extents_type, OtherExtents>` is true.

22 *Mandates:* `padding_stride == dynamic_extent || other_padding_stride == dynamic_extent || padding_stride == other_padding_stride` is true.

23 *Preconditions:*

- (23.1) — If `extents_type::rank() > 1` is true and `padding_stride` does not equal `dynamic_extent`, then `other.stride(extents_type::rank() - 2)` equals the least multiple of `padding_stride` greater than or equal to `extents_type::index-cast(other.extent(OtherExtents::rank() - 1))`; and
- (23.2) — `other.required_span_size()` is representable as a value of type `index_type` (**[basic.fundamental]**).

24 *Effects:*

- (24.1) — Direct-non-list-initializes *inner-mapping* with:
 - `other.extents()`, if `extents_type::rank()` is zero or one; else,
 - `other.extents().extent(P_right)...`, `other.stride(extents_type::rank() - 2)`; and
- (24.2) — if `extents_type::rank()` is zero, value-initializes *unpadded-extent*; else, direct-non-list-initializes *unpadded-extent* with `other.extents().extent(extents_type::rank() - 1)`.

25 *Remarks:* The expression inside `explicit` is equivalent to: `extents_type::rank() > 1 && (padding_stride == dynamic_extent || other_padding_stride == dynamic_extent)`.

```
template<size_t other_padding_stride, class OtherExtents>
constexpr explicit(! is_convertible_v<OtherExtents, extents_type>)
    mapping(const
        layout_left_padded<other_padding_stride>::mapping<OtherExtents>&
        other) noexcept;
```

26 *Constraints:*

- (26.1) — `extents_type::rank()` equals zero or one, and
- (26.2) — `is_constructible_v<extents_type, OtherExtents>` is true.

27 *Preconditions:* `other.required_span_size()` is representable as a value of type `index_type` (**[basic.fundamental]**).

28 *Effects:*

- (28.1) — Direct-non-list-initializes *inner-mapping* with `other.extents()`; and
- (28.2) — if `extents_type::rank()` is zero, value-initializes *unpadded-extent*; else, initializes *unpadded-extent* with `other.extents().extent(0)`.

[Note: Neither mapping uses the padding stride in the rank-0 or rank-1 case, so the padding stride does not affect either the constraints or the preconditions. – end note]


```
constexpr extents_type extents() const noexcept;
```

29 *Effects:*

- (29.1) — If `extents_type::rank()` is zero, equivalent to return `extents_type{}`;
- (29.2) — Otherwise, equivalent to return `extents_type(inner-mapping.extent(P_right)...,unpadded-extent.extent(extents_type::rank() - 1))`;

```
constexpr std::array<index_type, extents_type::rank()>  
strides() const noexcept;
```

30 *Effects:* Equivalent to return `inner-mapping.strides()`;

```
constexpr index_type required_span_size() const noexcept;
```

31 *Effects:* Equivalent to return `inner-mapping.required_span_size()`;

```
template<class... Indices>  
constexpr size_t operator()(Indices... idxs) const noexcept;
```

32 *Constraints:*

- (32.1) — `sizeof...(Indices) == Extents::rank()` is true,
- (32.2) — `(is_convertible_v<Indices, index_type> && ...)` is true, and
- (32.3) — `(is_nothrow_constructible<index_type, Indices> && ...)` is true.

33 *Precondition:* `extents_type::index-cast(i)` is a multidimensional index in `extents()` ([mdspan.overview]).

34 *Effects:* Let `P` be a parameter pack such that `is_same_v<index_sequence_for<Indices...>, index_sequence<P...>>` is true. Equivalent to: return `((static_cast<index_type>(i) * stride(P)) + ... + 0)`;

[*Note:* Effects are also equivalent to return `inner-mapping(idxs...)`; but only after the Precondition has been applied. – end note]

```
static constexpr bool is_always_exhaustive() noexcept;
```

35 *Returns:*

- (35.1) — If `extents_type::rank()` equals zero or one, true;
- (35.2) — else, if neither `inner-mapping-type::static_extent(extents_type::rank() - 1)` nor `extents_type::static_extent(extents_type::rank() - 1)` equal `dynamic_extent`, then `inner-mapping-type::static_extent(extents_type::rank() - 1) == extents_type::static_extent(extents_type::rank() - 1)`;
- (35.3) — otherwise, false.

```
constexpr bool is_exhaustive() const noexcept;
```

36 *Returns:*

- (36.1) — If `extents_type::rank()` equals zero, then true;
- (36.2) — else, `inner-mapping.extent(extents_type::rank() - 1) == unpadded-
extent.extent(extents_type::rank() - 1)`.

```
constexpr index_type stride(rank_type r) const noexcept;
```

37 *Effects:* Equivalent to return `inner-mapping.stride(r)`;

```
template<size_t other_padding_stride, class OtherExtents>  
friend constexpr bool operator==(  
    const mapping& x,  
    const typename  
        layout_right_padding<other_padding_stride>::mapping<OtherExtents>& y)  
noexcept;
```

38 *Constraints:* `OtherExtents::rank() == extents_type::rank()` is true.

39 *Returns:* true if

- (39.1) — `x.extents() == y.extents()` is true; and
- (39.2) — if `extents_type::rank() > 1` is true, then `x.stride(extents_type::rank() - 2) == y.stride(extents_type::rank() - 2)` is true.

§ 4.3 Layout specializations of `submdspan_mapping` [`mdspan.submdspan.mapping`]

At the top of Section  [`mdspan.submdspan.mapping`] (“Layout specializations of `submdspan_mapping`”), before paragraph 1, add the following to the end of the synopsis of specializations.

```
template<class Extents, std::size_t padding_stride, class...  
    SliceSpecifiers>  
constexpr auto submdspan_mapping(  
    const layout_left_padded<padding_stride>::template mapping<Extents>&  
        src,  
    SliceSpecifiers ... slices) -> /* see below */;  
  
template<class Extents, std::size_t padding_stride, class...  
    SliceSpecifiers>  
constexpr auto submdspan_mapping(  
    const layout_right_padded<padding_stride>::template mapping<Extents>&  
        src,  
    SliceSpecifiers ... slices) -> /* see below */;
```

In paragraph 7 (the “Returns” clause) of Section  [`mdspan.submdspan.mapping`] (“Layout specializations of `submdspan_mapping`”), replace (7.3) (the `layout_stride` fall-back return type) with the following.

- (9.4) `submdspan_mapping_result{layout_left_padded<Extents::static_extent(0)>::mapping(sub_ext, src.extent(0)), offset}` if
- `decltype(src)::layout_type` is `layout_left`; and
 - for each k in the range $[1, \text{SubExtents}::\text{rank}()-1]$, S_k is `full_extent_t`; and
 - `is_convertible_v< $S_0$ , tuple<index_type, index_type>>` is true; and
 - for k equal to `SubExtents::rank()-1`, `is_convertible_v< $S_k$ , tuple<index_type, index_type>> || is_convertible_v< $S_k$ , full_extent_t>` is true; otherwise
- (9.5) `submdspan_mapping_result{layout_right_padded<Extents::static_extent(0)>::template mapping(sub_ext, src.extent(0)), offset}` if
- `decltype(src)::layout_type` is `layout_right`; and
 - for each k in the range $[\text{Extents}::\text{rank}() - \text{SubExtents}::\text{rank}() + 1, \text{Extents}::\text{rank}() - 1]$, S_k is `full_extent_t`; and
 - for k equal to `Extents::rank() - 1` `is_convertible_v< $S_k$ , tuple<index_type, index_type>>` is true; and
 - for k equal to `SubExtents::rank() - SubExtents::rank()`, `is_convertible_v< $S_k$ , tuple<index_type, index_type>> || is_convertible_v< $S_k$ , full_extent_t>` is true; otherwise
- (9.6) `submdspan_mapping_result{layout_left_padded<dynamic_extent>::mapping(sub_ext, src.extent(0)), offset}` if
- `decltype(src)::layout_type` is a specialization of `layout_left_padded`; and
 - for each k in the range $[1, \text{SubExtents}::\text{rank}()-1]$, S_k is `full_extent_t`; and
 - `is_convertible_v< $S_0$ , tuple<index_type, index_type>> || is_convertible_v< $S_0$ , full_extent_t>` is true; and
 - for k equal to `SubExtents::rank()-1`, `is_convertible_v< $S_k$ , tuple<index_type, index_type>> || is_convertible_v< $S_k$ , full_extent_t>` is true; otherwise
- (9.7) `submdspan_mapping_result{layout_right_padded<dynamic_extent>::template mapping(sub_ext, src.extent(0)), offset}` if
- `decltype(src)::layout_type` is a specialization of `layout_right_padded`; and
 - for each k in the range $[\text{Extents}::\text{rank}() - \text{SubExtents}::\text{rank}() + 1, \text{Extents}::\text{rank}() - 1]$, S_k is `full_extent_t`; and
 - for k equal to `Extents::rank() - 1` `is_convertible_v< $S_k$ , tuple<index_type, index_type>> || is_convertible_v< $S_k$ , full_extent_t>` is true; and

— for k equal to SubExtents::rank() - SubExtents::rank(), is_convertible_v<S_k, tuple<index_type, index_type>> || is_convertible_v<S_k, full_extent_t> is true; otherwise

(9.8) submdspan_mapping_result{layout_stride::mapping(sub_ext, sub_strides), offset}.

§ 4.4 Layout specializations of submdspan_offset [mdspan.submdspan.offset]

At the top of Section  [mdspan.submdspan.offset] (“Layout specializations of submdspan_offset”), before paragraph 1, add the following to the end of the synopsis of specializations. (Note that all the specializations of submdspan_offset share the same wording.)

```
template<class Extents, std::size_t padding_stride, class...
    SliceSpecifiers>
constexpr size_t submdspan_offset(
    const layout_left_padded<padding_stride>::template mapping<Extents>&
        src,
    SliceSpecifiers ... slices);

template<class Extents, std::size_t padding_stride, class...
    SliceSpecifiers>
constexpr size_t submdspan_offset(
    const layout_right_padded<padding_stride>::template mapping<Extents>&
        src,
    SliceSpecifiers ... slices);
```